

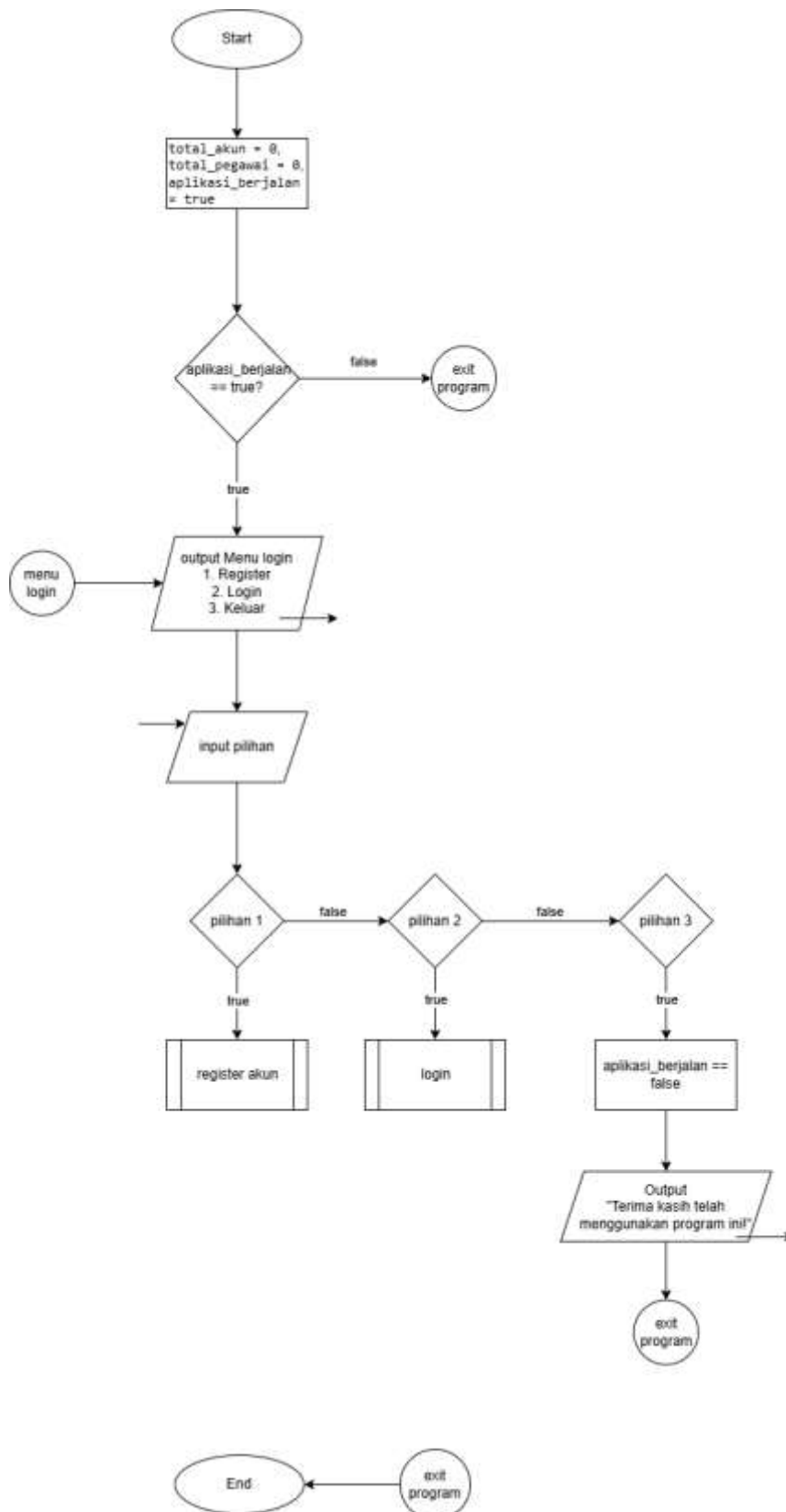
LAPORAN PRAKTIKUM
POSTTEST (3)
ALGORITMA PEMROGRAMAN LANJUT



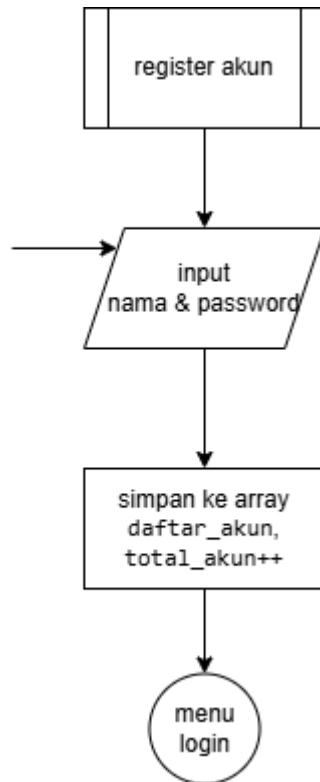
Disusun oleh:
Antung Hissyam (2509106092)
Kelas (C1 '25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2025

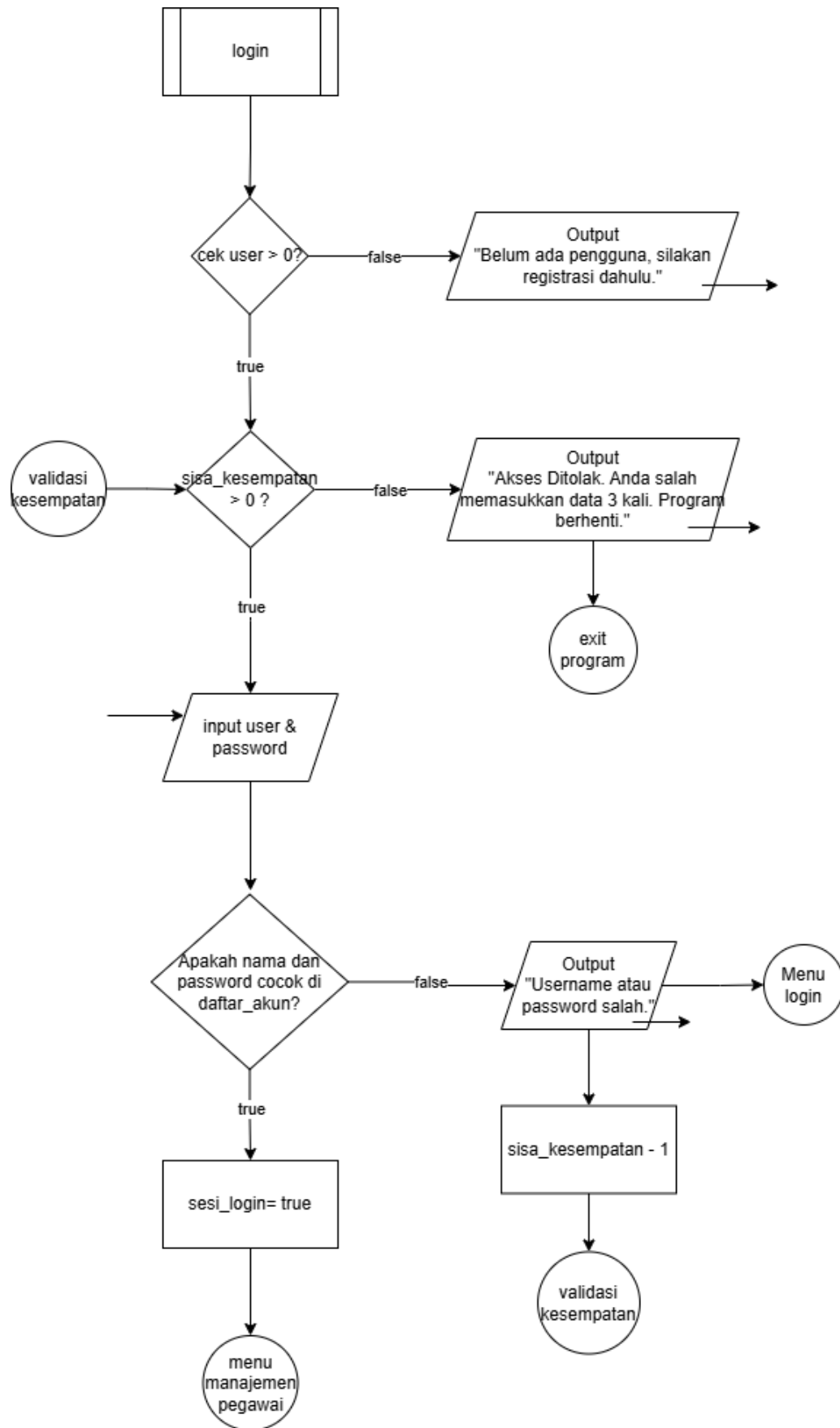
1. Flowchart



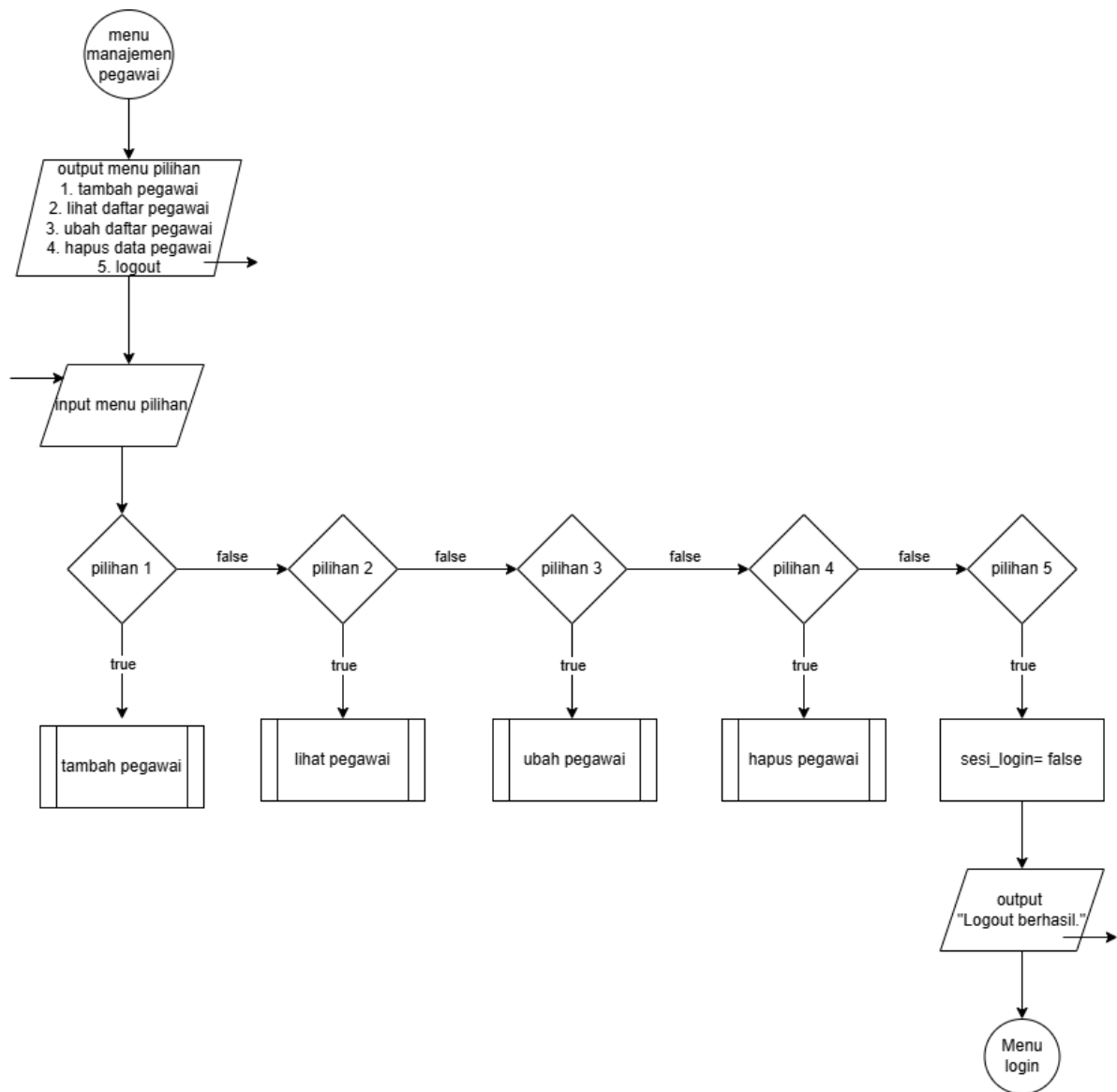
Gambar 1.1 Flowchart Menu Login.



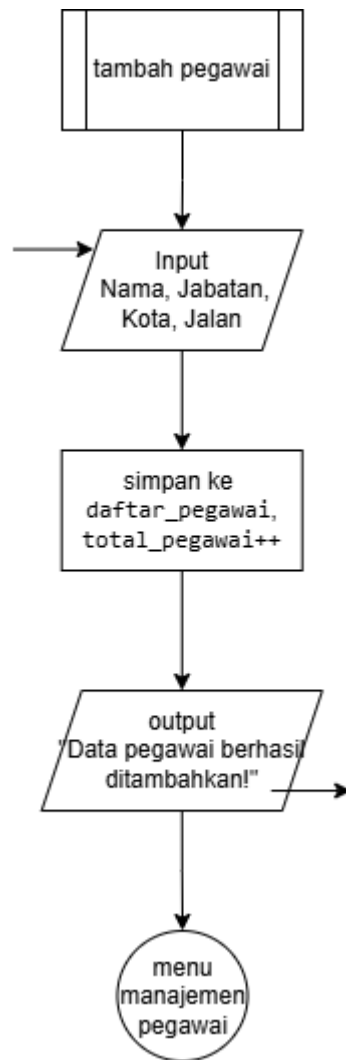
Gambar 1.2 Flowchart Fungsi Register Akun.



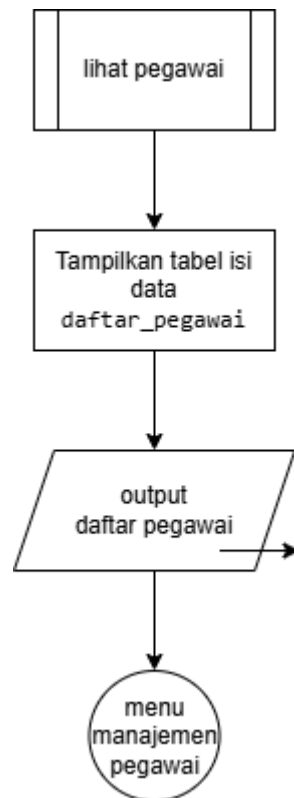
Gambar 1.3 Flowchart Fungsi Login.



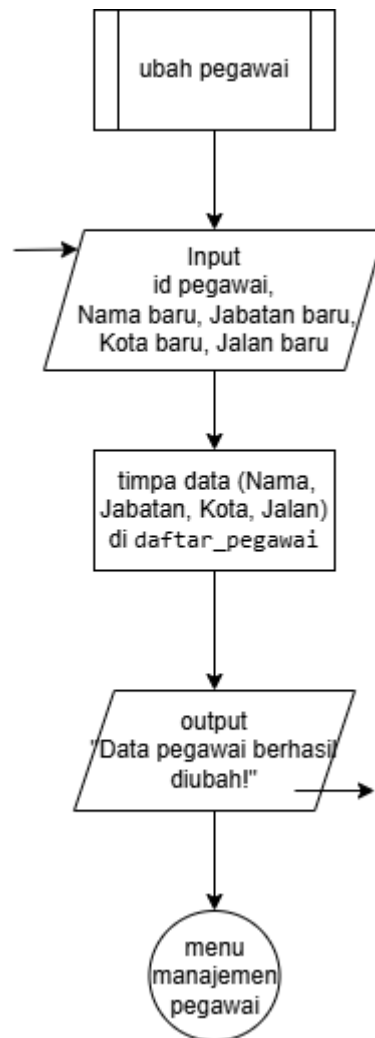
Gambar 1.4 Flowchart Menu CRUD.



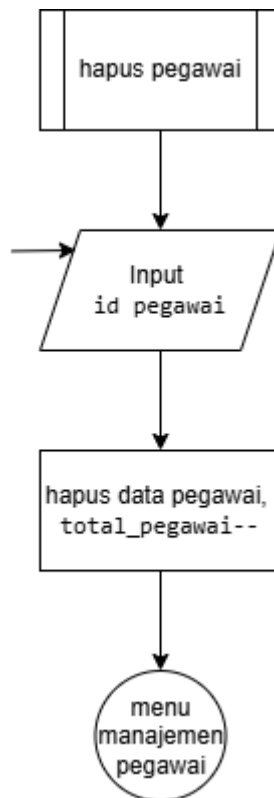
Gambar 1.5 Flowchart Fungsi Tambah Pegawai (Create).



Gambar 1.6 Flowchart Fungsi Lihat Pegawai (Read).



Gambar 1.7 Flowchart Fungsi Ubah Pegawai (Update).



Gambar 1.3 Flowchart Fungsi Hapus Pegawai (Delete).

Penjelasan Alur Flowchart (Algoritma Program Manajemen Pegawai)

Flowchart di atas merepresentasikan program Manajemen Pegawai yang sudah dipecah menjadi modul-modul sub-program (Prosedur dan Fungsi). Program utama hanya bertugas mengatur perulangan menu dan logika percabangan. Setiap kali pengguna memilih suatu opsi, alur tidak memproses data secara langsung di dalam program utama, melainkan memanggil simbol **Predefined Process (Sub-program)** seperti **registerAkun()**, **loginAkun()**, **tambahPegawai()**, hingga **hapusPegawai()**. Nilai kembalian (return value) dari fungsi **loginAkun()** digunakan sebagai kondisi decision untuk memberikan akses menuju menu manajemen CRUD. Pemecahan ini membuat alur diagram utama menjadi jauh lebih rapi dan ringkas.

2. Deskripsi Singkat Program

Program "Sistem Manajemen Pegawai Kebab Cendana" versi Post-Test 3 ini merupakan hasil refaktor dari program sebelumnya. Tujuan utamanya adalah mengimplementasikan modularitas menggunakan Prosedur (Void) dan Fungsi. Seluruh operasi CRUD dan autentikasi telah dipisahkan dari fungsi **main()** menggunakan sistem parameter dan Pass by Reference untuk memanipulasi isi array dan memori secara dinamis. Selain itu, program ini juga

menerapkan fitur lanjutan berupa Fungsi Overloading (fungsi dengan nama yang sama namun parameter berbeda untuk mencetak header menu) dan Fungsi Rekursif (untuk fitur hitung mundur sebelum program ditutup sepenuhnya).

3. Source Code

A. Penerapan Prosedur (Pass by Reference) & Parameter Array

Setiap operasi CRUD dipisahkan menjadi prosedur void. Variabel total_pegawai dilempar menggunakan pass by reference (&) agar perubahannya berdampak pada memori asli di fungsi main().

```
void tambahPegawai(Pegawai daftar_pegawai[], int &total_pegawai) {  
    cout << "\n--- TAMBAH PEGAWAI ---\n";  
    cout << "Nama Pegawai : ";  
    cin.ignore();  
    getline(cin, daftar_pegawai[total_pegawai].nama_pegawai);  
    // (lanjutan input struct...)  
    total_pegawai++;  
    cout << "[+] Data pegawai berhasil ditambahkan!\n";  
}
```

B. Penerapan Fungsi dengan Tipe Kembalian (Return Value)

Sistem login dipisahkan menggunakan fungsi bertipe bool. Jika validasi password pada array berhasil, fungsi akan mengembalikan nilai true untuk memberikan izin masuk ke program utama.

```
bool loginAkun(Akun daftar_akun[], int total_akun) {  
    // (logika pengecekan batas 3x percobaan dan input kredensial)  
    for (int i = 0; i < total_akun; i++) {  
        if (input_nama == daftar_akun[i].nama_pengguna &&  
            input_password == daftar_akun[i].password_pengguna) {  
            cout << "\n[+] Login Berhasil! Selamat datang, " <<  
            input_nama << ".\n";  
            return true;  
        }  
    }  
}
```

```
    return false;
}
```

C. Penerapan Fungsi Overloading

Terdapat dua fungsi bernama `tampilkanHeader()` yang memiliki parameter berbeda (tanpa parameter dan parameter string). Hal ini memungkinkan program memanggil nama fungsi yang sama untuk mencetak desain tabel header yang berbeda.

```
// implementasi overloading
void tampilkanHeader() {
    cout << "\n=====\\n";
    cout << "  SISTEM PEGAWAI KEBAB CENDANA (ADMIN)  \\n";
    cout << "=====\\n";
}

void tampilkanHeader(string sub_judul) {
    cout << "\n=====\\n";
    cout << "          " << sub_judul << "          \\n";
    cout << "=====\\n";
}
```

D. Penerapan Fungsi Rekursif

Fungsi `prosesKeluar` memanggil dirinya sendiri secara berulang untuk membuat efek hitung mundur di terminal sebelum memberhentikan program.

```
void prosesKeluar(int detik) {
    if (detik == 0) {
        cout << "Terima kasih telah menggunakan aplikasi ini!\\n";
        return;
    }
    cout << "Aplikasi akan tertutup dalam " << detik << "...\\n";
    prosesKeluar(detik - 1);
}
```

4. Hasil Output

A. Output Menu Register dan Login Berhasil

Tampilan ini menunjukkan simulasi saat pengguna pertama kali mendaftarkan akun, kemudian berhasil melakukan login menggunakan akun tersebut.

```
=====
                        SISTEM PEGAWAI KEBAB CENDANA
=====

--- MENU AUTENTIKASI ---
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 1

--- REGISTER ---
Masukkan Nama (Tanpa Spasi) : antung
Masukkan Password           : 092
[+] Akun berhasil didaftarkan! Silakan Login.

--- MENU AUTENTIKASI ---
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 2

--- LOGIN ---
Masukkan Nama      : antung
Masukkan Password  : 092

[+] Login Berhasil! Selamat datang, antung.

=====
                        MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: █
```

Gambar 4.1 Output Register dan Login Berhasil

B. Output Gagal Login 3 Kali (Akses Ditolak)

Tampilan ini membuktikan bahwa fitur pembatasan akses berjalan dengan baik. Jika pengguna salah menginput data sebanyak 3 kali, program otomatis dihentikan.

```

--- MENU AUTENTIKASI ---
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 2

--- LOGIN ---
Masukkan Nama      : aweawd
Masukkan Password  : wda
[!] Nama atau Password salah! Sisa percobaan: 2

Masukkan Nama      : wad
Masukkan Password  : dwa
[!] Nama atau Password salah! Sisa percobaan: 1

Masukkan Nama      : wad
Masukkan Password  : adw

[X] Anda gagal login 3 kali. Program berhenti otomatis.

```

Gambar 4.2 Output Login Gagal dan Program Berhenti

C. Output Tambah Pegawai (Create)

Tampilan ini menunjukkan ketika pengguna (admin) berhasil masuk ke menu CRUD dan melakukan penambahan data pegawai beserta alamatnya (Nested Struct).

```

=====
              MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 1

--- TAMBAH PEGAWAI ---
Nama Pegawai : antung
Jabatan      : ceo
Kota         : samarinda
Jalan        : cendana
[+] Data pegawai berhasil ditambahkan!

```

Gambar 4.3 Output Tambah Data Pegawai

D. Output Tampilkan Data (Read)

Tampilan ini membuktikan bahwa data yang diinputkan sebelumnya berhasil disimpan di dalam array of struct dan ditampilkan dengan format tabel yang rapi.

```

=====
MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 2

--- DAFTAR PEGAWAI KEBAB CENDANA ---
-----
No.      | Nama                | Jabatan                | Alamat (Kota, Jalan)
-----
1        | antung              | ceo                    | samarinda, cendana
-----

```

Gambar 4.4 Output Tabel Daftar Pegawai

E. Output Ubah dan Hapus Data (Update & Delete)

Tampilan ini menunjukkan proses pencarian data berdasarkan nomor indeks. Data pegawai akan diubah jika memilih opsi 3, atau dihapus jika memilih opsi 4.

```

=====
MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 3

--- UBAH DATA PEGAWAI ---
Masukkan nomor pegawai yang akan diubah: 1
Masukkan nama baru : antoeng
Masukkan jabatan baru: ceo muda aselole
Masukkan kota baru : balikpapan
Masukkan jalan baru : sudirman
[+] Data berhasil diubah.

=====
MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 4

--- HAPUS DATA PEGAWAI ---
Masukkan nomor pegawai yang akan dihapus: 1
[+] Pegawai berhasil dihapus.

```

Gambar 4.5 Output Ubah dan Hapus Data

F. Output Pilihan Tidak Valid dan Keluar

Tampilan ini menunjukkan ketika pengguna menginput angka di luar daftar menu (contoh: angka 6), program akan menampilkan peringatan dan mengulang menu.

```
=====
MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 6
[!] Pilihan tidak valid!
```

Gambar 4.6 Output Validasi Menu

G. Output Exit Program

Tampilan ini menunjukkan ketika pengguna memilih angka 4, program akan berhenti.

```
=====
SISTEM PEGAWAI KEBAB CENDANA (ADMIN)
=====
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 3
Aplikasi akan tertutup dalam 3...
Aplikasi akan tertutup dalam 2...
Aplikasi akan tertutup dalam 1...
Terima kasih telah menggunakan aplikasi ini!
```

Gambar 4.7 Output Program Selesai

5. Langkah-langkah GIT

5.1 GIT Add

```
● PS E:\Coding\praktikum-apl\post-test\post-test-apl-1> git add .
```

Perintah git add . berfungsi untuk menambahkan semua file yang baru dibuat atau file yang mengalami modifikasi ke dalam *staging area*.

5.2 GIT Commit

```
● PS E:\Coding\praktikum-apl\post-test\post-test-apl-1> git commit -m "PT-1"
[main f5782f9] PT-1
2 files changed, 105 insertions(+)
create mode 100644 post-test/post-test-apl-1/2509106092-AntungHissyam-PT-1.cpp
create mode 100644 post-test/post-test-apl-1/2509106092-AntungHissyam-PT-1.exe
```

Perintah git commit berfungsi untuk merekam dan menyimpan secara permanen perubahan file yang sudah berada di *staging area* ke dalam riwayat repositori lokal.

5.3 GIT Push

```
PS E:\Coding\praktikum-apl\post-test\post-test-apl-1> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 448.22 KiB | 6.90 MiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
```

Perintah git push berfungsi untuk mengunggah semua *commit* atau perubahan yang ada di repositori lokal (komputer) menuju ke repositori *remote* yang ada di Github.